กลุ่ม 3 (Group 3) | รายวิชา Bioinformatics Programming Project II (20%) | กำหนดส่ง 17 พฤศจิกายน 2569
สมาชิก: **[1]** ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1) · **[2]** ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2) · **[3]** ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3) · **[4]** ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4) · **[5]** ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `main_pipeline.py (ส่วนที่ 1/4)`

ฟังก์ชันหลัก (Main Function Orchestrator) — ผู้รับผิดชอบ: สมาชิกร่วมกันทุกคน

```
"""
Bioinformatics Programming Project II – กลุ่ม 3
โมดูลฟังก์ชันหลัก (Main Function Orchestrator): main_pipeline.py

หัวข้อวิจัย:
การทำนายความไวและการดื้อยาปฏิชีวนะของแบคทีเรีย Escherichia coli จากข้อมูลทางพันธุกรรมและข้อมูลฟีโนไทป์คลินิก
(Predicting Antibiotic Susceptibility and Resistance in E. coli from Genomic Determinants and Clinical Phenotypes)
"""

from __future__ import annotations

import argparse
import sys
from pathlib import Path
from typing import Any

PROJECT2_ROOT = Path(__file__).resolve().parent
if str(PROJECT2_ROOT.parent) not in sys.path:
    sys.path.insert(0, str(PROJECT2_ROOT.parent))

from project2.modules.mod1_clinical_phenome import load_and_profile_clinical_resistance
from project2.modules.mod2_genome_fetcher import fetch_and_parse_isolate_genomes
from project2.modules.mod3_amr_gene_scanner import scan_amr_genes_and_mutations
from project2.modules.mod4_genotype_phenotype_map import map_genetic_determinants_to_drugs
from project2.modules.mod5_resistance_predictor import predict_susceptibility_and_evaluate


def run_amr_prediction_pipeline(
    clinical_csv_path: str | Path = PROJECT2_ROOT / "data/raw/Ecoli_antibiotic_resistance_data.csv",
    isolate_fasta_path: str | Path = PROJECT2_ROOT / "data/raw/ena_ncbi_ecoli_isolates.fasta",
    isolate_metadata_path: str | Path = PROJECT2_ROOT / "data/raw/ena_ncbi_isolate_metadata.json",
    reference_amr_fasta_path: str | Path = PROJECT2_ROOT / "data/raw/ncbi_amr_reference_genes.fasta",
    output_dir: str | Path = PROJECT2_ROOT / "data/output",
    fetch_online: bool = False,
) -> dict[str, Any]:
    """
    ฟังก์ชันหลัก (Main Function): ควบคุมลำดับการทำงานของฟังก์ชันย่อยทั้ง 5 โมดูลเพื่อวิเคราะห์และทำนาย
    ความไวและการดื้อยาปฏิชีวนะ (S/NS และค่า MIC) ของเชื้อ Escherichia coli

    ข้อมูลเข้า (Input):
        clinical_csv_path (str | Path): พาธไฟล์ CSV ข้อมูลฟีโนไทป์คลินิก (GradLab 574,239 แถว)
        isolate_fasta_path (str | Path): พาธไฟล์ FASTA ลำดับเบสจีโนมของสายพันธุ์ E. coli จาก NCBI/ENA
        isolate_metadata_path (str | Path): พาธไฟล์ JSON Metadata ของสายพันธุ์จาก NCBI/ENA
        reference_amr_fasta_path (str | Path): พาธไฟล์ FASTA ยีนดื้อยาอ้างอิงจาก NCBI/ENA (11 ยีนหลัก)
        output_dir (str | Path): โฟลเดอร์ปลายทางสำหรับบันทึกตารางผลการทำนาย CSV และรายงาน JSON
        fetch_online (bool): เปิด/ปิดการดึงข้อมูลเพิ่มเติมจาก ENA REST API (ค่าเริ่มต้น False ใช้ข้อมูล Offline)

    การดำเนินงาน (Action):
        1. เรียกใช้ฟังก์ชันย่อยที่ 1 (load_and_profile_clinical_resistance) เพื่อวิเคราะห์ระบาดวิทยาคลินิกและค่า MIC
        2. เรียกใช้ฟังก์ชันย่อยที่ 2 (fetch_and_parse_isolate_genomes) เพื่อโหลดลำดับจีโนมและตรวจสอบคุณภาพ (%GC, ความยาว)
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `main_pipeline.py (ส่วนที่ 2/4)`

ฟังก์ชันหลัก (Main Function Orchestrator) — ผู้รับผิดชอบ: สมาชิกร่วมกันทุกคน

```
  51 |         3. เรียกใช้ฟังก์ชันย่อยที่ 3 (scan_amr_genes_and_mutations) เพื่อค้นหายีนดื้อยาและตรวจหา Point Mutations
  52 |         4. เรียกใช้ฟังก์ชันย่อยที่ 4 (map_genetic_determinants_to_drugs) เพื่อคำนวณคะแนนกลไกการดื้อยาต่อยา 5 ชนิด
  53 |         5. เรียกใช้ฟังก์ชันย่อยที่ 5 (predict_susceptibility_and_evaluate) เพื่อทำนาย S/NS, คาดการณ์ค่า MIC,
  54 |            จำแนกสายพันธุ์ MDR และประเมินความแม่นยำ (Accuracy, Sensitivity, Specificity, F1-Score)
  55 |
  56 |     ผลลัพธ์ (Output):
  57 |         dict[str, Any]: ผลลัพธ์รวมของการทำงานทั้ง 5 โมดูล พร้อมสถิติการประเมินความแม่นยำและพาธไฟล์ส่งออก
  58 |     """
  59 |     # ขั้นตอนที่ 1: วิเคราะห์ข้อมูลฟีโนไทป์คลินิกและระบาดวิทยา (Module 1)
  60 |     clinical_profile = load_and_profile_clinical_resistance(csv_path=clinical_csv_path)
  61 |
  62 |     # ขั้นตอนที่ 2: โหลดลำดับเบสจีโนมและตรวจสอบคุณภาพลำดับนิวคลีโอไทด์ (Module 2)
  63 |     isolate_genomes = fetch_and_parse_isolate_genomes(
  64 |         fasta_path=isolate_fasta_path,
  65 |         metadata_path=isolate_metadata_path,
  66 |         fetch_online=fetch_online,
  67 |     )
  68 |
  69 |     # ขั้นตอนที่ 3: สแกนหายีนดื้อยาและตรวจสอบการกลายพันธุ์ระดับกรดอะมิโน (Module 3)
  70 |     scan_results = scan_amr_genes_and_mutations(
  71 |         isolate_genomes=isolate_genomes,
  72 |         reference_fasta_path=reference_amr_fasta_path,
  73 |     )
  74 |
  75 |     # ขั้นตอนที่ 4: เชื่อมโยงกลไกการดื้อยาระดับโมเลกุลเข้ากับยาปฏิชีวนะทั้ง 5 ชนิด (Module 4)
  76 |     mapped_profiles = map_genetic_determinants_to_drugs(
  77 |         scan_results=scan_results,
  78 |         clinical_profile=clinical_profile,
  79 |     )
  80 |
  81 |     # ขั้นตอนที่ 5: ทำนายสถานะ S/NS, ค่า MIC, สายพันธุ์ MDR และประเมินความแม่นยำ (Module 5)
  82 |     evaluation_summary = predict_susceptibility_and_evaluate(
  83 |         mapped_profiles=mapped_profiles,
  84 |         clinical_profile=clinical_profile,
  85 |         isolate_genomes=isolate_genomes,
  86 |         output_dir=output_dir,
  87 |     )
  88 |
  89 |     return evaluation_summary
  90 |
  91 |
  92 | def main(argv: list[str] | None = None) -> int:
  93 |     """ส่วนติดต่อบรรทัดคำสั่ง (CLI Entrypoint) สำหรับรันโปรแกรมหลักของ Project II"""
  94 |     parser = argparse.ArgumentParser(
  95 |         description="Bioinformatics Programming Project II (Group 3): E. coli AMR Genomic & Clinical Prediction Pipeline"
  96 |     )
  97 |     parser.add_argument(
  98 |         "--clinical-csv",
  99 |         default=str(PROJECT2_ROOT / "data/raw/Ecoli_antibiotic_resistance_data.csv"),
 100 |         help="Path to clinical resistance CSV dataset",
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `main_pipeline.py (ส่วนที่ 3/4)`

ฟังก์ชันหลัก (Main Function Orchestrator) — ผู้รับผิดชอบ: สมาชิกร่วมกันทุกคน

```
101 |     )
102 |     parser.add_argument(
103 |         "--isolates-fasta",
104 |         default=str(PROJECT2_ROOT / "data/raw/ena_ncbi_ecoli_isolates.fasta"),
105 |         help="Path to offline E. coli isolate genomes FASTA",
106 |     )
107 |     parser.add_argument(
108 |         "--isolates-meta",
109 |         default=str(PROJECT2_ROOT / "data/raw/ena_ncbi_isolate_metadata.json"),
110 |         help="Path to offline NCBI/ENA isolate metadata JSON",
111 |     )
112 |     parser.add_argument(
113 |         "--ref-amr-fasta",
114 |         default=str(PROJECT2_ROOT / "data/raw/ncbi_amr_reference_genes.fasta"),
115 |         help="Path to offline NCBI/ENA AMR reference genes FASTA",
116 |     )
117 |     parser.add_argument(
118 |         "--output-dir",
119 |         default=str(PROJECT2_ROOT / "data/output"),
120 |         help="Directory for output predictions CSV and evaluation JSON",
121 |     )
122 |     parser.add_argument(
123 |         "--online",
124 |         action="store_true",
125 |         help="Enable optional live query verification against EBI ENA REST API",
126 |     )
127 |
128 |     args = parser.parse_args(argv)
129 |     result = run_amr_prediction_pipeline(
130 |         clinical_csv_path=args.clinical_csv,
131 |         isolate_fasta_path=args.isolates_fasta,
132 |         isolate_metadata_path=args.isolates_meta,
133 |         reference_amr_fasta_path=args.ref_amr_fasta,
134 |         output_dir=args.output_dir,
135 |         fetch_online=args.online,
136 |     )
137 |
138 |     clin = result["clinical_dataset_overview"]
139 |     geno = result["genomic_cohort_overview"]
140 |     ov = result["evaluation_metrics"]["overall"]
141 |
142 |     print("=" * 78)
143 |     print("BIOINFORMATICS PROGRAMMING PROJECT II — GROUP 3 (E. COLI AMR PIPELINE)")
144 |     print("=" * 78)
145 |     print(
146 |         f"[Module 1] Clinical Phenome Analyzed : {clin['total_records_analyzed']:,} records "
147 |         f"({clin['unique_clinical_isolates']:,} unique isolates, "
148 |         f"MDR rate = {clin['clinical_mdr_summary']['mdr_rate']*100:.2f}%)"
149 |     )
150 |     print(
```

ไฟล์โมดูล: `main_pipeline.py (ส่วนที่ 4/4)`

ฟังก์ชันหลัก (Main Function Orchestrator) — ผู้รับผิดชอบ: สมาชิกร่วมกันทุกคน

```
151 |         f"[Module 2-3] Genomic Cohort Scanned  : {geno['total_isolates_sequenced']} E. coli genomes "
152 |         f"({geno['total_drug_test_predictions']} drug-isolate pairs)"
153 |     )
154 |     print(
155 |         f"[Module 4-5] Prediction Performance  : Accuracy = {ov['accuracy']*100:.2f}% | "
156 |         f"Sensitivity (NS) = {ov['sensitivity_ns']*100:.2f}% | "
157 |         f"Specificity (S) = {ov['specificity_s']*100:.2f}% | "
158 |         f"Exact MIC Match = {ov['exact_mic_concordance_rate']*100:.2f}%"
159 |     )
160 |     print(f"[Output CSV] {result['output_files']['predictions_csv']}")
161 |     print(f"[Output JSON] {result['output_files']['summary_json']}")
162 |     print("=" * 78)
163 |     return 0
164 |
165 |
166 | if __name__ == "__main__":
167 |     raise SystemExit(main())
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod1_clinical_phenome.py (ส่วนที่ 1/6)`

โมดูลที่ 1 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1)

```
"""
Module 1: mod1_clinical_phenome.py
ฟังก์ชันย่อยที่ 1 – การคัดกรองและวิเคราะห์ระบาดวิทยาคลินิก ค่า MIC และฟีโนไทป์การดื้อยา (S/NS)
จากฐานข้อมูล Ecoli_antibiotic_resistance_data.csv (GradLab)

ผู้รับผิดชอบ: สมาชิกคนที่ 1 (กลุ่ม 3)
"""

from __future__ import annotations

import csv
import math
from collections import Counter, defaultdict
from pathlib import Path
from typing import Any

TARGET_ANTIBIOTICS: tuple[str, ...] = ("AMC", "AMP", "CIP", "NIT", "TET")


def calculate_mic_percentiles(mic_values: list[float]) -> dict[str, float]:
    """
    คำนวณค่าสถิติของความเข้มข้นต่ำสุดที่ยับยั้งเชื้อได้ (MIC50, MIC90, ค่าเฉลี่ยเรขาคณิต และพิสัย)

    ข้อมูลเข้า (Input):
        mic_values (list[float]): รายการค่า MIC (หน่วย ug/mL) ที่มากกว่า 0

    การดำเนินงาน (Action):
        เรียงลำดับค่า MIC จากน้อยไปมาก คำนวณเปอร์เซ็นไทล์ที่ 50 (MIC50) และ 90 (MIC90)
        พร้อมคำนวณค่าเฉลี่ยเรขาคณิต (Geometric Mean MIC) ด้วยฐานลอการิทึมธรรมชาติ

    ผลลัพธ์ (Output):
        dict[str, float]: พจนานุกรมประกอบด้วย min_mic, mic50, mic90, max_mic และ geometric_mean_mic
    """
    if not mic_values:
        raise ValueError("mic_values ต้องมีข้อมูลอย่างน้อย 1 ค่า")

    cleaned = sorted(float(v) for v in mic_values if float(v) > 0.0)
    if not cleaned:
        raise ValueError("mic_values ต้องมีค่ามากกว่า 0")

    n = len(cleaned)
    idx50 = min(n - 1, max(0, math.ceil(0.50 * n) - 1))
    idx90 = min(n - 1, max(0, math.ceil(0.90 * n) - 1))
    log_sum = sum(math.log(v) for v in cleaned)
    geom_mean = math.exp(log_sum / n)

    return {
        "min_mic": round(cleaned[0], 4),
        "mic50": round(cleaned[idx50], 4),
        "mic90": round(cleaned[idx90], 4),
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod1_clinical_phenome.py (ส่วนที่ 2/6)`

โมดูลที่ 1 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1)

```
 51 |         "max_mic": round(cleaned[-1], 4),
 52 |         "geometric_mean_mic": round(geom_mean, 4),
 53 |     }
 54 |
 55 |
 56 | def summarize_isolate_mdr_patterns(
 57 |     isolate_drug_classes_ns: dict[str, set[str]],
 58 |     total_unique_isolates: int,
 59 | ) -> dict[str, Any]:
 60 |     """
 61 |     วิเคราะห์รูปแบบการดื้อยาหลายขนาน (Multi-Drug Resistance: MDR) ในระดับสายพันธุ์ (Isolate-level)
 62 |     โดยนิยาม MDR คือการไม่ไวต่อยา (NS) อย่างน้อย 3 กลุ่มยาปฏิชีวนะขึ้นไป
 63 |
 64 |     ข้อมูลเข้า (Input):
 65 |         isolate_drug_classes_ns (dict[str, set[str]]): แผนที่จาก isolate_ID ไปยังเซตของ drug_class ที่เป็น NS
 66 |         total_unique_isolates (int): จำนวนสายพันธุ์ทั้งหมดในชุดข้อมูล
 67 |
 68 |     การดำเนินงาน (Action):
 69 |         นับจำนวนกลุ่มยาที่แต่ละสายพันธุ์ดื้อยา จำแนกเป็น Pan-susceptible (0 กลุ่ม),
 70 |         Resistant 1-2 กลุ่ม และ MDR (>= 3 กลุ่มยา)
 71 |
 72 |     ผลลัพธ์ (Output):
 73 |         dict[str, Any]: สรุปจำนวนและร้อยละของสายพันธุ์ที่เป็น MDR และการกระจายตามจำนวนกลุ่มยาที่ดื้อ
 74 |     """
 75 |     if total_unique_isolates <= 0:
 76 |         raise ValueError("total_unique_isolates ต้องมากกว่า 0")
 77 |
 78 |     class_count_dist: Counter[int] = Counter()
 79 |     mdr_isolates = 0
 80 |
 81 |     for _, ns_classes in isolate_drug_classes_ns.items():
 82 |         k = len(ns_classes)
 83 |         class_count_dist[k] += 1
 84 |         if k >= 3:
 85 |             mdr_isolates += 1
 86 |
 87 |     pan_susceptible = total_unique_isolates - sum(class_count_dist.values())
 88 |     class_count_dist[0] += max(0, pan_susceptible)
 89 |
 90 |     return {
 91 |         "total_unique_isolates": total_unique_isolates,
 92 |         "pan_susceptible_isolates": class_count_dist[0],
 93 |         "pan_susceptible_rate": round(class_count_dist[0] / total_unique_isolates, 4),
 94 |         "mdr_isolates_count": mdr_isolates,
 95 |         "mdr_rate": round(mdr_isolates / total_unique_isolates, 4),
 96 |         "ns_drug_class_distribution": {
 97 |             str(k): class_count_dist[k] for k in sorted(class_count_dist)
 98 |         },
 99 |     }
100 |
```

ไฟล์โมดูล: `modules/mod1_clinical_phenome.py (ส่วนที่ 3/6)`

โมดูลที่ 1 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1)

```
101 |
102 | def load_and_profile_clinical_resistance(
103 |     csv_path: str | Path,
104 |     target_drugs: tuple[str, ...] = TARGET_ANTIBIOTICS,
105 | ) -> dict[str, Any]:
106 |     """
107 |     ฟังก์ชันย่อยหลักที่ 1: อ่านตารางข้อมูลฟีโนไทป์คลินิกของเชื้อ E. coli แบบ Streaming
108 |     คำนวณอัตราความไว (S) และการดื้อยา (NS), ค่า MIC50/MIC90, ความเสี่ยงตามแหล่งติดเชื้อ/ประเภทผู้ป่วย
109 |     และสัดส่วนเชื้อดื้อยาหลายขนาน (MDR)
110 |
111 |     ข้อมูลเข้า (Input):
112 |         csv_path (str | Path): พาธไปยังไฟล์ Ecoli_antibiotic_resistance_data.csv
113 |         target_drugs (tuple[str, ...]): รหัสยาปฏิชีวนะที่ต้องการวิเคราะห์ (ค่าเริ่มต้น: AMC, AMP, CIP, NIT, TET)
114 |
115 |     การดำเนินงาน (Action):
116 |         1. ตรวจสอบไฟล์และคอลัมน์มาตรฐานของข้อมูลคลินิก
117 |         2. อ่านแถวข้อมูลแบบประหยัดหน่วยความจำ คัดกรองตามรหัสยาที่กำหนด
118 |         3. เรียกใช้ฟังก์ชัน calculate_mic_percentiles() และ summarize_isolate_mdr_patterns()
119 |         4. คำนวณความน่าจะเป็นพื้นฐานของการดื้อยา (Empirical Prior NS Probability) แยกตามตัวยาและประเภทผู้ป่วย
120 |
121 |     ผลลัพธ์ (Output):
122 |         dict[str, Any]: โครงสร้างข้อมูล ClinicalResistanceProfile สำหรับส่งต่อให้ Module 4 และ Module 5
123 |     """
124 |     path = Path(csv_path)
125 |     if not path.is_file():
126 |         raise FileNotFoundError(f"ไม่พบไฟล์ข้อมูลคลินิก CSV: {path}")
127 |
128 |     required_cols = {
129 |         "isolate_ID",
130 |         "year",
131 |         "hospital",
132 |         "patient_type",
133 |         "site_of_infection",
134 |         "drug_code",
135 |         "drug_name",
136 |         "drug_class",
137 |         "MIC",
138 |         "phenotype",
139 |     }
140 |
141 |     target_set = set(target_drugs)
142 |     total_rows = 0
143 |     all_isolates: set[str] = set()
144 |     drug_meta: dict[str, dict[str, str]] = {}
145 |     drug_phenotypes: dict[str, Counter[str]] = defaultdict(Counter)
146 |     drug_mics: dict[str, list[float]] = defaultdict(list)
147 |     drug_mics_by_pheno: dict[str, dict[str, list[float]]] = defaultdict(
148 |         lambda: {"S": [], "NS": []}
149 |     )
150 |     patient_type_ns: dict[str, dict[str, Counter[str]]] = defaultdict(
```

ไฟล์โมดูล: `modules/mod1_clinical_phenome.py (ส่วนที่ 4/6)`

โมดูลที่ 1 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1)

```
151 |         lambda: defaultdict(Counter)
152 |     )
153 |     site_counts: Counter[str] = Counter()
154 |     year_ns_counts: dict[str, Counter[str]] = defaultdict(Counter)
155 |     isolate_ns_classes: dict[str, set[str]] = defaultdict(set)
156 |
157 |     with path.open("r", encoding="utf-8", newline="") as handle:
158 |         reader = csv.DictReader(handle)
159 |         if not reader.fieldnames or not required_cols.issubset(set(reader.fieldnames)):
160 |             raise ValueError(f"คอลัมน์ในไฟล์ CSV ไม่ครบตามมาตรฐาน: {reader.fieldnames}")
161 |
162 |         for row in reader:
163 |             drug_code = (row.get("drug_code") or "").strip()
164 |             if drug_code not in target_set:
165 |                 continue
166 |
167 |             phenotype = (row.get("phenotype") or "").strip().upper()
168 |             if phenotype not in ("S", "NS"):
169 |                 continue
170 |
171 |             try:
172 |                 mic_val = float(row.get("MIC") or 0.0)
173 |             except ValueError:
174 |                 continue
175 |             if mic_val <= 0.0:
176 |                 continue
177 |
178 |             total_rows += 1
179 |             isolate_id = (row.get("isolate_ID") or "").strip()
180 |             drug_class = (row.get("drug_class") or "").strip()
181 |             patient_type = (row.get("patient_type") or "unknown").strip()
182 |             site = (row.get("site_of_infection") or "unknown").strip()
183 |             year = (row.get("year") or "unknown").strip()
184 |
185 |             all_isolates.add(isolate_id)
186 |             drug_meta[drug_code] = {
187 |                 "drug_name": (row.get("drug_name") or "").strip(),
188 |                 "drug_class": drug_class,
189 |             }
190 |             drug_phenotypes[drug_code][phenotype] += 1
191 |             drug_mics[drug_code].append(mic_val)
192 |             drug_mics_by_pheno[drug_code][phenotype].append(mic_val)
193 |             patient_type_ns[drug_code][patient_type][phenotype] += 1
194 |             site_counts[site] += 1
195 |             year_ns_counts[year][phenotype] += 1
196 |
197 |             if phenotype == "NS":
198 |                 isolate_ns_classes[isolate_id].add(drug_class)
199 |
200 |     if total_rows == 0:
```

ไฟล์โมดูล: `modules/mod1_clinical_phenome.py (ส่วนที่ 5/6)`

โมดูลที่ 1 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1)

```
201 |         raise ValueError("ไม่พบรายการข้อมูลที่ถูกต้องในไฟล์ CSV")
202 |
203 |     drug_profiles: dict[str, Any] = {}
204 |     for drug_code in sorted(drug_mics):
205 |         s_count = drug_phenotypes[drug_code]["S"]
206 |         ns_count = drug_phenotypes[drug_code]["NS"]
207 |         tested = s_count + ns_count
208 |         mic_stats = calculate_mic_percentiles(drug_mics[drug_code])
209 |
210 |         s_mics = drug_mics_by_pheno[drug_code]["S"]
211 |         ns_mics = drug_mics_by_pheno[drug_code]["NS"]
212 |         s_median = calculate_mic_percentiles(s_mics)["mic50"] if s_mics else 0.0
213 |         ns_median = calculate_mic_percentiles(ns_mics)["mic50"] if ns_mics else 0.0
214 |
215 |         inpatient_c = patient_type_ns[drug_code].get("inpatient", Counter())
216 |         outpatient_c = patient_type_ns[drug_code].get("outpatient", Counter())
217 |         inpatient_tot = inpatient_c["S"] + inpatient_c["NS"]
218 |         outpatient_tot = outpatient_c["S"] + outpatient_c["NS"]
219 |
220 |         drug_profiles[drug_code] = {
221 |             "drug_code": drug_code,
222 |             "drug_name": drug_meta[drug_code]["drug_name"],
223 |             "drug_class": drug_meta[drug_code]["drug_class"],
224 |             "total_tested": tested,
225 |             "susceptible_count": s_count,
226 |             "non_susceptible_count": ns_count,
227 |             "susceptible_rate": round(s_count / tested, 4),
228 |             "non_susceptible_rate": round(ns_count / tested, 4),
229 |             "inpatient_ns_rate": round(inpatient_c["NS"] / inpatient_tot, 4)
230 |             if inpatient_tot
231 |             else 0.0,
232 |             "outpatient_ns_rate": round(outpatient_c["NS"] / outpatient_tot, 4)
233 |             if outpatient_tot
234 |             else 0.0,
235 |             "median_s_mic": s_median,
236 |             "median_ns_mic": ns_median,
237 |             **mic_stats,
238 |         }
239 |
240 |     mdr_summary = summarize_isolate_mdr_patterns(
241 |         isolate_ns_classes, len(all_isolates)
242 |     )
243 |
244 |     return {
245 |         "source_csv": str(path),
246 |         "total_records_analyzed": total_rows,
247 |         "unique_isolates": len(all_isolates),
248 |         "drug_profiles": drug_profiles,
249 |         "infection_site_distribution": dict(site_counts.most_common()),
250 |         "mdr_summary": mdr_summary,
```

ไฟล์โมดูล: `modules/mod1_clinical_phenome.py (ส่วนที่ 6/6)`

โมดูลที่ 1 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 1 (6XXXXXXXX-1)

```
251 |         "years_covered": sorted(year_ns_counts.keys()),
252 |     }
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod2_genome_fetcher.py (ส่วนที่ 1/5)`

โมดูลที่ 2 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2)

```
"""
Module 2: mod2_genome_fetcher.py
ฟังก์ชันย่อยที่ 2 — การดึงและอ่านข้อมูลลำดับเบสจีโนม (FASTA) และ Metadata จากฐานข้อมูล NCBI Datasets และ EBI ENA Browser
พร้อมการตรวจสอบคุณภาพลำดับนิวคลีโอไทด์ (Sequence Quality Control)

ผู้รับผิดชอบ: สมาชิกคนที่ 2 (กลุ่ม 3)
"""

from __future__ import annotations

import json
import urllib.parse
import urllib.request
from pathlib import Path
from typing import Any

VALID_IUPAC_DNA: frozenset[str] = frozenset("ACGTNRYSWKMBDHV")


def parse_fasta_records(fasta_path: str | Path) -> list[dict[str, str]]:
    """
    อ่านไฟล์ลำดับนิวคลีโอไทด์รูปแบบ FASTA และแยกส่วน Header Metadata ออกจากลำดับเบส

    ข้อมูลเข้า (Input):
        fasta_path (str | Path): พาธไปยังไฟล์ FASTA (.fasta หรือ .fa)

    การดำเนินงาน (Action):
        อ่านบรรทัดทั้งหมดในไฟล์ แยกบรรทัดที่ขึ้นต้นด้วย '>' เป็นส่วนหัว (Header)
        และรวมบรรทัดลำดับเบสเข้าด้วยกันเป็นตัวพิมพ์ใหญ่

    ผลลัพธ์ (Output):
        list[dict[str, str]]: รายการพจนานุกรมที่มีคีย์ record_id, header และ sequence
    """
    path = Path(fasta_path)
    if not path.is_file():
        raise FileNotFoundError(f"ไม่พบไฟล์ FASTA: {path}")

    records: list[dict[str, str]] = []
    current_header: str | None = None
    seq_chunks: list[str] = []

    with path.open("r", encoding="utf-8") as handle:
        for raw_line in handle:
            line = raw_line.strip()
            if not line:
                continue
            if line.startswith(">"):
                if current_header is not None:
                    seq = "".join(seq_chunks).upper()
                    rec_id = current_header.split()[0].split("|")[0]
```

ไฟล์โมดูล: `modules/mod2_genome_fetcher.py (ส่วนที่ 2/5)`

โมดูลที่ 2 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2)

```
                    records.append(
                        {
                            "record_id": rec_id,
                            "header": current_header,
                            "sequence": seq,
                        }
                    )
                current_header = line[1:].strip()
                seq_chunks = []
            else:
                seq_chunks.append(line)

    if current_header is not None:
        seq = "".join(seq_chunks).upper()
        rec_id = current_header.split()[0].split("|")[0]
        records.append(
            {
                "record_id": rec_id,
                "header": current_header,
                "sequence": seq,
            }
        )

    if not records:
        raise ValueError(f"ไม่พบเรคคอร์ด FASTA ที่ถูกต้องในไฟล์: {path}")

    return records


def compute_sequence_qc_metrics(sequence: str) -> dict[str, Any]:
    """
    คำนวณค่าคุณภาพของลำดับนิวคลีโอไทด์ ได้แก่ ความยาว (bp), สัดส่วน %GC, สัดส่วนเบสกำกวม (%N)
    และสถานะการผ่านเกณฑ์คุณภาพพื้นฐานสำหรับจีโนม E. coli

    ข้อมูลเข้า (Input):
        sequence (str): ลำดับนิวคลีโอไทด์ (A, C, G, T, N)

    การดำเนินงาน (Action):
        ตรวจสอบอักขระเบส คำนวณความยาวรวม นับจำนวน G และ C เพื่อหาค่า %GC
        และตรวจสอบว่าค่า %N ไม่เกิน 5% และความยาวมากกว่า 500 bp

    ผลลัพธ์ (Output):
        dict[str, Any]: ค่าสถิติ length_bp, gc_percent, ambiguous_n_percent และ qc_passed
    """
    seq = sequence.strip().upper()
    if not seq:
        raise ValueError("ลำดับเบส (sequence) ต้องไม่เป็นค่าว่าง")

    invalid_chars = set(seq) - VALID_IUPAC_DNA
    if invalid_chars:
```

ไฟล์โมดูล: `modules/mod2_genome_fetcher.py (ส่วนที่ 3/5)`

โมดูลที่ 2 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2)

```
101 |         raise ValueError(f"พบอักขระที่ไม่ใช่ลำดับนิวคลีโอไทด์ IUPAC: {sorted(invalid_chars)}")
102 |
103 |     length = len(seq)
104 |     g_count = seq.count("G")
105 |     c_count = seq.count("C")
106 |     n_count = seq.count("N")
107 |     atgc_count = seq.count("A") + seq.count("T") + g_count + c_count
108 |
109 |     gc_percent = ((g_count + c_count) / atgc_count * 100.0) if atgc_count > 0 else 0.0
110 |     n_percent = (n_count / length * 100.0) if length > 0 else 0.0
111 |     qc_passed = (length >= 300) and (n_percent <= 5.0) and (35.0 <= gc_percent <= 65.0)
112 |
113 |     return {
114 |         "length_bp": length,
115 |         "gc_percent": round(gc_percent, 2),
116 |         "ambiguous_n_percent": round(n_percent, 4),
117 |         "qc_passed": qc_passed,
118 |     }
119 |
120 |
121 | def query_ena_ncbi_online_metadata(accession_id: str, timeout: int = 10) -> dict[str, Any]:
122 |     """
123 |     ดึงข้อมูล Metadata ของสายพันธุ์หรือยีนจาก EBI ENA Portal REST API (ใช้เมื่อเปิดโหมดออนไลน์)
124 |
125 |     ข้อมูลเข้า (Input):
126 |         accession_id (str): รหัส Accession ของ ENA/NCBI (เช่น KC844055 หรือ MK564156)
127 |         timeout (int): เวลาสูงสุดในการรอคำตอบจากเซิร์ฟเวอร์ (วินาที)
128 |
129 |     การดำเนินงาน (Action):
130 |         ส่งคำขอ HTTP GET ไปยัง EBI ENA Portal API และแปลงข้อมูล JSON กลับมาเป็นพจนานุกรม
131 |
132 |     ผลลัพธ์ (Output):
133 |         dict[str, Any]: ข้อมูล accession, scientific_name, strain และ base_count จาก ENA
134 |     """
135 |     clean_acc = accession_id.split(".")[0].strip()
136 |     if not clean_acc:
137 |         raise ValueError("accession_id ต้องไม่เป็นค่าว่าง")
138 |
139 |     query = urllib.parse.quote(f'accession="{clean_acc}"')
140 |     url = (
141 |         "https://www.ebi.ac.uk/ena/portal/api/search?result=sequence"
142 |         f"&query={query}&fields=accession,description,scientific_name,strain,base_count&format=json"
143 |     )
144 |     with urllib.request.urlopen(url, timeout=timeout) as response:
145 |         payload = json.loads(response.read().decode("utf-8"))
146 |         if isinstance(payload, list) and payload:
147 |             return payload[0]
148 |     return {"accession": clean_acc, "status": "not_found"}
149 |
150 |
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod2_genome_fetcher.py (ส่วนที่ 4/5)`

โมดูลที่ 2 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2)

```
151 | def fetch_and_parse_isolate_genomes(
152 |     fasta_path: str | Path,
153 |     metadata_path: str | Path,
154 |     fetch_online: bool = False,
155 | ) -> list[dict[str, Any]]:
156 |     """
157 |     ฟังก์ชันย่อยหลักที่ 2: โหลดข้อมูลลำดับเบสจีโนมของสายพันธุ์ E. coli จากไฟล์ FASTA แบบ Offline
158 |     เชื่อมโยงกับ Metadata จาก NCBI Datasets และ EBI ENA Browser พร้อมคำนวณค่า QC ของทุกสายพันธุ์
159 |
160 |     ข้อมูลเข้า (Input):
161 |         fasta_path (str | Path): พาธไปยังไฟล์ ena_ncbi_ecoli_isolates.fasta
162 |         metadata_path (str | Path): พาธไปยังไฟล์ ena_ncbi_isolate_metadata.json
163 |         fetch_online (bool): หากเป็น True จะตรวจสอบข้อมูลเพิ่มเติมจาก ENA API (ค่าเริ่มต้น False เพื่อทำงานแบบ Offline)
164 |
165 |     การดำเนินงาน (Action):
166 |         1. เรียกใช้ parse_fasta_records() เพื่ออ่านลำดับเบสของทุกสายพันธุ์
167 |         2. อ่านไฟล์ JSON Metadata ที่ระบุรหัส NCBI Assembly, ENA Accession, BioSample และข้อมูลคลินิก
168 |         3. เรียกใช้ compute_sequence_qc_metrics() เพื่อตรวจสอบคุณภาพลำดับเบสของแต่ละสายพันธุ์
169 |         4. ผสานข้อมูลลำดับเบส คุณภาพจีโนม และ Metadata เข้าเป็นโครงสร้างเดียวกัน
170 |
171 |     ผลลัพธ์ (Output):
172 |         list[dict[str, Any]]: รายการข้อมูลสายพันธุ์ (IsolateGenome) ที่ผ่านการตรวจสอบคุณภาพพร้อมส่งให้ Module 3
173 |     """
174 |     meta_file = Path(metadata_path)
175 |     if not meta_file.is_file():
176 |         raise FileNotFoundError(f"ไม่พบไฟล์ Metadata JSON: {meta_file}")
177 |
178 |     fasta_records = parse_fasta_records(fasta_path)
179 |     raw_meta = json.loads(meta_file.read_text(encoding="utf-8"))
180 |     meta_by_id: dict[str, dict[str, Any]] = {
181 |         item["isolate_id"]: item for item in raw_meta.get("isolates", [])
182 |     }
183 |
184 |     isolate_genomes: list[dict[str, Any]] = []
185 |     for rec in fasta_records:
186 |         iso_id = rec["record_id"]
187 |         seq = rec["sequence"]
188 |         qc = compute_sequence_qc_metrics(seq)
189 |         meta = meta_by_id.get(iso_id, {})
190 |
191 |         online_info = None
192 |         if fetch_online and meta.get("ena_accession"):
193 |             try:
194 |                 online_info = query_ena_ncbi_online_metadata(meta["ena_accession"])
195 |             except Exception:
196 |                 online_info = {"status": "offline_fallback"}
197 |
198 |         isolate_genomes.append(
199 |             {
200 |                 "isolate_id": iso_id,
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod2_genome_fetcher.py (ส่วนที่ 5/5)`

โมดูลที่ 2 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 2 (6XXXXXXXX-2)

```
201 |                 "clinical_id": meta.get("clinical_id", "UNKNOWN"),
202 |                 "strain": meta.get("strain", rec["header"]),
203 |                 "ncbi_assembly": meta.get("ncbi_assembly", "N/A"),
204 |                 "ena_accession": meta.get("ena_accession", "N/A"),
205 |                 "biosample": meta.get("biosample", "N/A"),
206 |                 "year": meta.get("year"),
207 |                 "hospital": meta.get("hospital", "N/A"),
208 |                 "patient_type": meta.get("patient_type", "N/A"),
209 |                 "site_of_infection": meta.get("site_of_infection", "N/A"),
210 |                 "sequence": seq,
211 |                 "qc_metrics": qc,
212 |                 "known_phenotypes": meta.get("phenotypes", {}),
213 |                 "online_verification": online_info,
214 |             }
215 |         )
216 |
217 |     return isolate_genomes
```

ไฟล์โมดูล: `modules/mod3_amr_gene_scanner.py (ส่วนที่ 1/6)`

โมดูลที่ 3 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3)

```
"""
Module 3: mod3_amr_gene_scanner.py
ฟังก์ชันย่อยที่ 3 — การสแกนหายีนดื้อยาปฏิชีวนะ (Acquired AMR Genes) และตรวจหาการกลายพันธุ์ระดับจุด
(Chromosomal Point Mutations ใน QRDR ของ gyrA/parC และยีน nitroreductase nfsA/nfsB)

ผู้รับผิดชอบ: สมาชิกคนที่ 3 (กลุ่ม 3)
"""

from __future__ import annotations

from pathlib import Path
from typing import Any

from .mod2_genome_fetcher import parse_fasta_records

BACTERIAL_CODON_TABLE: dict[str, str] = {
    "TTT": "F", "TTC": "F", "TTA": "L", "TTG": "L",
    "CTT": "L", "CTC": "L", "CTA": "L", "CTG": "L",
    "ATT": "I", "ATC": "I", "ATA": "I", "ATG": "M",
    "GTT": "V", "GTC": "V", "GTA": "V", "GTG": "V",
    "TCT": "S", "TCC": "S", "TCA": "S", "TCG": "S",
    "CCT": "P", "CCC": "P", "CCA": "P", "CCG": "P",
    "ACT": "T", "ACC": "T", "ACA": "T", "ACG": "T",
    "GCT": "A", "GCC": "A", "GCA": "A", "GCG": "A",
    "TAT": "Y", "TAC": "Y", "TAA": "*", "TAG": "*",
    "CAT": "H", "CAC": "H", "CAA": "Q", "CAG": "Q",
    "AAT": "N", "AAC": "N", "AAA": "K", "AAG": "K",
    "GAT": "D", "GAC": "D", "GAA": "E", "GAG": "E",
    "TGT": "C", "TGC": "C", "TGA": "*", "TGG": "W",
    "CGT": "R", "CGC": "R", "CGA": "R", "CGG": "R",
    "AGT": "S", "AGC": "S", "AGA": "R", "AGG": "R",
    "GGT": "G", "GGC": "G", "GGA": "G", "GGG": "G",
}

ACQUIRED_AMR_GENES: tuple[str, ...] = (
    "blaTEM-1",
    "blaCTX-M-15",
    "blaOXA-1",
    "ampC",
    "qnrS1",
    "tetA",
    "tetB",
)

CORE_RESISTANCE_LOCI: dict[str, list[tuple[int, str, str]]] = {
    "gyrA": [
        (83, "S", "Quinolone target DNA gyrase subunit A QRDR codon 83"),
        (87, "D", "Quinolone target DNA gyrase subunit A QRDR codon 87"),
    ],
    "parC": [
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod3_amr_gene_scanner.py (ส่วนที่ 2/6)`

โมดูลที่ 3 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3)

```
        (80, "S", "Quinolone target topoisomerase IV subunit A QRDR codon 80"),
        (84, "E", "Quinolone target topoisomerase IV subunit A QRDR codon 84"),
    ],
    "nfsA": [
        (15, "R", "Nitrofuran reductase NfsA active-site codon 15"),
    ],
    "nfsB": [
        (44, "Q", "Nitrofuran reductase NfsB catalytic domain codon 44"),
        (94, "W", "Nitrofuran reductase NfsB FMN-binding codon 94"),
    ],
}


def translate_bacterial_cds(nucleotide_seq: str) -> str:
    """
    แปลรหัสพันธุกรรมจากลำดับนิวคลีโอไทด์ (DNA Coding Sequence) เป็นลำดับกรดอะมิโนตามรหัสพันธุกรรมแบคทีเรีย (Table 11)

    ข้อมูลเข้า (Input):
        nucleotide_seq (str): ลำดับเบส DNA ของยีน (ความยาวเป็นพหุคูณของ 3)

    การดำเนินงาน (Action):
        อ่านทีละ 3 เบส (Codon) จากตำแหน่งเริ่มต้นและแปลงเป็นรหัสกรดอะมิโน 1 ตัวอักษร

    ผลลัพธ์ (Output):
        str: ลำดับกรดอะมิโนที่ได้จากการแปลรหัส
    """
    seq = nucleotide_seq.strip().upper()
    if len(seq) < 3:
        raise ValueError("ความยาวลำดับเบสต้องมีอย่างน้อย 3 นิวคลีโอไทด์ (1 Codon)")

    codons = [seq[i : i + 3] for i in range(0, len(seq) - 2, 3)]
    return "".join(BACTERIAL_CODON_TABLE.get(c, "X") for c in codons)


def align_and_score_gene_hit(
    contig_seq: str,
    ref_seq: str,
    kmer_size: int = 15,
) -> dict[str, Any]:
    """
    ค้นหาตำแหน่งยีนอ้างอิงในจีโนมด้วย K-mer Seed Anchoring และคำนวณค่า Sequence Identity และ Coverage

    ข้อมูลเข้า (Input):
        contig_seq (str): ลำดับนิวคลีโอไทด์ของจีโนมสายพันธุ์ที่ต้องการตรวจสอบ
        ref_seq (str): ลำดับนิวคลีโอไทด์ของยีนดื้อยาอ้างอิงจาก NCBI/ENA
        kmer_size (int): ขนาด K-mer สำหรับค้นหาจุดเริ่มต้นของการจัดเรียงลำดับ (ค่าเริ่มต้น 15 bp)

    การดำเนินงาน (Action):
        ค้นหาตำแหน่ง K-mer แรกหรือ K-mer ตรงกลางของยีนอ้างอิงใน contig_seq
        จากนั้นเปรียบเทียบเบสต่อเบสตลอดความยาวของยีนอ้างอิงเพื่อคำนวณ %Identity และ %Coverage
```

ไฟล์โมดูล: `modules/mod3_amr_gene_scanner.py (ส่วนที่ 3/6)`

โมดูลที่ 3 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3)

```
101 |
102 |     ผลลัพธ์ (Output):
103 |         dict[str, Any]: พจนานุกรมประกอบด้วย found, start_pos, end_pos, identity และ coverage
104 |     """
105 |     if not contig_seq or not ref_seq:
106 |         raise ValueError("contig_seq และ ref_seq ต้องไม่เป็นค่าว่าง")
107 |
108 |     ref_len = len(ref_seq)
109 |     if len(contig_seq) < ref_len:
110 |         return {"found": False, "start_pos": -1, "end_pos": -1, "identity": 0.0, "coverage": 0.0}
111 |
112 |     seed_offsets = [0, ref_len // 4, ref_len // 2]
113 |     candidate_starts: set[int] = set()
114 |
115 |     for offset in seed_offsets:
116 |         seed = ref_seq[offset : offset + kmer_size]
117 |         pos = contig_seq.find(seed)
118 |         while pos != -1:
119 |             cand = pos - offset
120 |             if 0 <= cand <= len(contig_seq) - ref_len:
121 |                 candidate_starts.add(cand)
122 |             pos = contig_seq.find(seed, pos + 1)
123 |
124 |     if not candidate_starts:
125 |         return {"found": False, "start_pos": -1, "end_pos": -1, "identity": 0.0, "coverage": 0.0}
126 |
127 |     best_start = -1
128 |     best_matches = -1
129 |     for start in sorted(candidate_starts):
130 |         sub = contig_seq[start : start + ref_len]
131 |         matches = sum(1 for a, b in zip(sub, ref_seq) if a == b)
132 |         if matches > best_matches:
133 |             best_matches = matches
134 |             best_start = start
135 |
136 |     identity = best_matches / ref_len
137 |     coverage = 1.0 if best_start >= 0 else 0.0
138 |     aligned_sub = contig_seq[best_start : best_start + ref_len]
139 |
140 |     return {
141 |         "found": True,
142 |         "start_pos": best_start,
143 |         "end_pos": best_start + ref_len,
144 |         "identity": round(identity, 4),
145 |         "coverage": round(coverage, 4),
146 |         "aligned_sequence": aligned_sub,
147 |     }
148 |
149 |
150 | def detect_qrdr_and_nitroreductase_mutations(
```

ไฟล์โมดูล: `modules/mod3_amr_gene_scanner.py (ส่วนที่ 4/6)`

โมดูลที่ 3 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3)

```
151 |     contig_seq: str,
152 |     reference_genes: dict[str, str],
153 | ) -> list[dict[str, Any]]:
154 |     """
155 |     ตรวจสอบการกลายพันธุ์ระดับกรดอะมิโน (Point Mutations / Nonsense Mutations) ในยีนเป้าหมายบนโครโมโซม
156 |     ได้แก่ gyrA (S83L, D87N/Y), parC (S80I, E84V), nfsA (R15C/R15*) และ nfsB (Q44*/W94*)
157 |
158 |     ข้อมูลเข้า (Input):
159 |         contig_seq (str): ลำดับเบสจีโนมของสายพันธุ์ E. coli
160 |         reference_genes (dict[str, str]): พจนานุกรมลำดับเบสยีนอ้างอิง Wild-type
161 |
162 |     การดำเนินงาน (Action):
163 |         ค้นหาตำแหน่งยีน gyrA, parC, nfsA และ nfsB ด้วย align_and_score_gene_hit()
164 |         แปลรหัสเป็นลำดับกรดอะมิโนด้วย translate_bacterial_cds() และเทียบกับกรดอะมิโน Wild-type ในตำแหน่งสำคัญ
165 |
166 |     ผลลัพธ์ (Output):
167 |         list[dict[str, Any]]: รายการการกลายพันธุ์ที่ตรวจพบพร้อมระบุรหัสการกลายพันธุ์ เช่น gyrA_S83L หรือ nfsA_R15*
168 |     """
169 |     detected_mutations: list[dict[str, Any]] = []
170 |
171 |     for gene_name, hotspots in CORE_RESISTANCE_LOCI.items():
172 |         ref_seq = reference_genes.get(gene_name)
173 |         if not ref_seq:
174 |             continue
175 |
176 |         hit = align_and_score_gene_hit(contig_seq, ref_seq)
177 |         if not hit["found"] or hit["identity"] < 0.85:
178 |             continue
179 |
180 |         isolate_aa = translate_bacterial_cds(hit["aligned_sequence"])
181 |         for codon_pos_1based, wt_aa, description in hotspots:
182 |             aa_idx = codon_pos_1based - 1
183 |             if aa_idx >= len(isolate_aa):
184 |                 continue
185 |             obs_aa = isolate_aa[aa_idx]
186 |             if obs_aa != wt_aa:
187 |                 codon_str = hit["aligned_sequence"][aa_idx * 3 : (aa_idx + 1) * 3]
188 |                 mut_type = "nonsense_truncation" if obs_aa == "*" else "missense_substitution"
189 |                 detected_mutations.append(
190 |                     {
191 |                         "gene": gene_name,
192 |                         "codon_position": codon_pos_1based,
193 |                         "wildtype_aa": wt_aa,
194 |                         "observed_aa": obs_aa,
195 |                         "observed_codon": codon_str,
196 |                         "mutation_label": f"{gene_name}_{wt_aa}{codon_pos_1based}{obs_aa}",
197 |                         "mutation_type": mut_type,
198 |                         "functional_region": description,
199 |                     }
200 |                 )
```

ไฟล์โมดูล: `modules/mod3_amr_gene_scanner.py (ส่วนที่ 5/6)`

โมดูลที่ 3 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3)

```
201 |
202 |     return detected_mutations
203 |
204 |
205 | def scan_amr_genes_and_mutations(
206 |     isolate_genomes: list[dict[str, Any]],
207 |     reference_fasta_path: str | Path,
208 |     min_identity: float = 0.90,
209 |     min_coverage: float = 0.85,
210 | ) -> dict[str, dict[str, Any]]:
211 |     """
212 |     ฟังก์ชันย่อยหลักที่ 3: สแกนลำดับจีโนมของทุกสายพันธุ์เพื่อค้นหายีนดื้อยาที่รับมาจากพลาสมิด (Acquired AMR Genes)
213 |     และการกลายพันธุ์ระดับโครโมโซม (Chromosomal Point Mutations)
214 |
215 |     ข้อมูลเข้า (Input):
216 |         isolate_genomes (list[dict[str, Any]]): รายการจีโนมสายพันธุ์จาก Module 2
217 |         reference_fasta_path (str | Path): พาธไปยังไฟล์ยีนดื้อยาอ้างอิง ncbi_amr_reference_genes.fasta
218 |         min_identity (float): ค่าความเหมือนขั้นต่ำของลำดับเบส (ค่าเริ่มต้น 0.90 หรือ 90%)
219 |         min_coverage (float): ค่าความครอบคลุมความยาวขั้นต่ำของยีน (ค่าเริ่มต้น 0.85 หรือ 85%)
220 |
221 |     การดำเนินงาน (Action):
222 |         1. โหลดลำดับเบสยีนอ้างอิงจาก NCBI/ENA ด้วยฟังก์ชัน parse_fasta_records() จาก Module 2
223 |         2. สำหรับแต่ละสายพันธุ์ เรียกใช้ align_and_score_gene_hit() เพื่อตรวจหายีนดื้อยา 7 ชนิด
224 |         3. เรียกใช้ detect_qrdr_and_nitroreductase_mutations() เพื่อตรวจหาการกลายพันธุ์ใน gyrA, parC, nfsA, nfsB
225 |         4. รวบรวมโปรไฟล์พันธุกรรมดื้อยาของแต่ละสายพันธุ์
226 |
227 |     ผลลัพธ์ (Output):
228 |         dict[str, dict[str, Any]]: แผนที่จาก isolate_id ไปยังผลการตรวจพบยีนดื้อยาและการกลายพันธุ์ทั้งหมด
229 |     """
230 |     if not (0.50 <= min_identity <= 1.0) or not (0.50 <= min_coverage <= 1.0):
231 |         raise ValueError("min_identity และ min_coverage ต้องอยู่ในช่วง 0.50 ถึง 1.00")
232 |
233 |     ref_records = parse_fasta_records(reference_fasta_path)
234 |     reference_genes: dict[str, str] = {
235 |         rec["record_id"]: rec["sequence"] for rec in ref_records
236 |     }
237 |     reference_headers: dict[str, str] = {
238 |         rec["record_id"]: rec["header"] for rec in ref_records
239 |     }
240 |
241 |     scan_results: dict[str, dict[str, Any]] = {}
242 |     for iso in isolate_genomes:
243 |         iso_id = iso["isolate_id"]
244 |         contig_seq = iso["sequence"]
245 |
246 |         acquired_hits: list[dict[str, Any]] = []
247 |         for gene_name in ACQUIRED_AMR_GENES:
248 |             ref_seq = reference_genes.get(gene_name)
249 |             if not ref_seq:
250 |                 continue
```

ไฟล์โมดูล: `modules/mod3_amr_gene_scanner.py (ส่วนที่ 6/6)`

โมดูลที่ 3 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 3 (6XXXXXXXX-3)

```
251 |             hit = align_and_score_gene_hit(contig_seq, ref_seq)
252 |             if (
253 |                 hit["found"]
254 |                 and hit["identity"] >= min_identity
255 |                 and hit["coverage"] >= min_coverage
256 |             ):
257 |                 acquired_hits.append(
258 |                     {
259 |                         "gene": gene_name,
260 |                         "identity": hit["identity"],
261 |                         "coverage": hit["coverage"],
262 |                         "start_pos": hit["start_pos"],
263 |                         "end_pos": hit["end_pos"],
264 |                         "reference_annotation": reference_headers.get(gene_name, ""),
265 |                     }
266 |                 )
267 |
268 |         point_mutations = detect_qrdr_and_nitroreductase_mutations(
269 |             contig_seq, reference_genes
270 |         )
271 |
272 |         scan_results[iso_id] = {
273 |             "isolate_id": iso_id,
274 |             "clinical_id": iso.get("clinical_id", "UNKNOWN"),
275 |             "strain": iso.get("strain", ""),
276 |             "acquired_amr_genes": acquired_hits,
277 |             "acquired_gene_names": [h["gene"] for h in acquired_hits],
278 |             "point_mutations": point_mutations,
279 |             "mutation_labels": [m["mutation_label"] for m in point_mutations],
280 |             "total_genetic_determinants": len(acquired_hits) + len(point_mutations),
281 |         }
282 |
283 |     return scan_results
```

BIOINFORMATICS PROGRAMMING PROJECT II · เอกสารชุดคำสั่งโปรแกรม (SOURCE CODE PDF)

ไฟล์โมดูล: `modules/mod4_genotype_phenotype_map.py (ส่วนที่ 1/5)`

โมดูลที่ 4 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4)

```
"""
Module 4: mod4_genotype_phenotype_map.py
ฟังก์ชันย่อยที่ 4 — การเชื่อมโยงตัวยีนดื้อยาและการกลายพันธุ์ระดับโมเลกุลเข้ากับกลไกการออกฤทธิ์ของยาปฏิชีวนะทั้ง 5 ชนิด
(AMC, AMP, CIP, NIT, TET) และการคำนวณคะแนนกลไกการดื้อยาระดับพันธุกรรม (Genotypic Resistance Mechanism Score)

ผู้รับผิดชอบ: สมาชิกคนที่ 4 (กลุ่ม 3)
"""

from __future__ import annotations

from typing import Any

CLSI_DRUG_BREAKPOINTS: dict[str, dict[str, Any]] = {
    "AMC": {
        "drug_name": "Amoxicillin/Clavulanate",
        "drug_class": "Penicillins",
        "s_max_mic": 8.0,
        "ns_min_mic": 16.0,
        "wildtype_mic": 4.0,
    },
    "AMP": {
        "drug_name": "Ampicillin",
        "drug_class": "Penicillins",
        "s_max_mic": 8.0,
        "ns_min_mic": 16.0,
        "wildtype_mic": 4.0,
    },
    "CIP": {
        "drug_name": "Ciprofloxacin",
        "drug_class": "Quinolones",
        "s_max_mic": 1.0,
        "ns_min_mic": 2.0,
        "wildtype_mic": 0.25,
    },
    "NIT": {
        "drug_name": "Nitrofurantoin",
        "drug_class": "Nitrofurans",
        "s_max_mic": 32.0,
        "ns_min_mic": 64.0,
        "wildtype_mic": 16.0,
    },
    "TET": {
        "drug_name": "Tetracycline",
        "drug_class": "Tetracyclines",
        "s_max_mic": 4.0,
        "ns_min_mic": 8.0,
        "wildtype_mic": 2.0,
    },
}

```

ไฟล์โมดูล: `modules/mod4_genotype_phenotype_map.py (ส่วนที่ 2/5)`

โมดูลที่ 4 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4)

```
 51 |
 52 | def evaluate_drug_mechanism_score(
 53 |     drug_code: str,
 54 |     detected_genes: set[str],
 55 |     detected_mutations: set[str],
 56 |     baseline_ns_rate: float = 0.20,
 57 | ) -> dict[str, Any]:
 58 |     """
 59 |     คำนวณคะแนนกลไกการดื้อยา (Genotypic Resistance Score: 0.00 - 1.00) และระบุกลไกระดับชีวโมเลกุล
 60 |     สำหรับยาปฏิชีวนะแต่ละชนิดจากยีนดื้อยาและการกลายพันธุ์ที่ตรวจพบในจีโนม
 61 |
 62 |     ข้อมูลเข้า (Input):
 63 |         drug_code (str): รหัสยาปฏิชีวนะ (AMC, AMP, CIP, NIT หรือ TET)
 64 |         detected_genes (set[str]): เซตของชื่อยีนดื้อยาที่ตรวจพบจาก Module 3
 65 |         detected_mutations (set[str]): เซตของรหัสการกลายพันธุ์ระดับจุดที่ตรวจพบจาก Module 3
 66 |         baseline_ns_rate (float): อัตราการดื้อยาพื้นฐานในประชากรคลินิกจาก Module 1
 67 |
 68 |     การดำเนินงาน (Action):
 69 |         ประเมินกฎทางชีวโมเลกุลของเชื้อ E. coli:
 70 |         - AMP: ดื้อยาเมื่อมีเอนไซม์ beta-lactamase ชนิดใดก็ได้ (blaTEM-1, blaCTX-M-15, blaOXA-1, ampC)
 71 |         - AMC: Clavulanate ยับยั้ง blaTEM-1 และ blaCTX-M-15 ได้บางส่วน (MIC 8 ug/mL = S) แต่จะดื้อยา (NS)
 72 |           เมื่อมี blaOXA-1 (Inhibitor-resistant oxacillinase) หรือ ampC (Cephalosporinase ที่ไม่ถูกยับยั้งโดย clavulanate)
 73 |         - CIP: ดื้อยาแบบขั้นบันได (Stepwise accumulation) จากการกลายพันธุ์ใน QRDR ของ gyrA (S83L, D87N/Y)
 74 |           ร่วมกับ parC (S80I, E84V) หรือมียีนพลาสมิด qnrS1 ร่วมกับ gyrA S83L
 75 |         - NIT: ดื้อยาเมื่อเกิดการกลายพันธุ์สูญเสียหน้าที่ในยีนเอนไซม์ nitroreductase ทั้ง nfsA และ nfsB
 76 |         - TET: ดื้อยาเมื่อมียีนปั๊มขับยาออกจากเซลล์ (Efflux pump) tetA หรือ tetB
 77 |
 78 |     ผลลัพธ์ (Output):
 79 |         dict[str, Any]: คะแนน genetic_score, active_determinants, molecular_mechanism และ estimated_fold_increase
 80 |     """
 81 |     if drug_code not in CLSI_DRUG_BREAKPOINTS:
 82 |         raise ValueError(f"ไม่รองรับรหัสยา: {drug_code}")
 83 |
 84 |     active_determinants: list[str] = []
 85 |     mechanisms: list[str] = []
 86 |     genetic_score = 0.05
 87 |     fold_increase = 1.0
 88 |
 89 |     if drug_code == "AMP":
 90 |         beta_lactamases = [
 91 |             g for g in ("blaTEM-1", "blaCTX-M-15", "blaOXA-1", "ampC") if g in detected_genes
 92 |         ]
 93 |         if beta_lactamases:
 94 |             active_determinants.extend(beta_lactamases)
 95 |             mechanisms.append("Enzymatic hydrolysis of aminopenicillin beta-lactam ring")
 96 |             genetic_score = 0.96
 97 |             fold_increase = 8.0
 98 |
 99 |     elif drug_code == "AMC":
100 |         if "ampC" in detected_genes:
```

ไฟล์โมดูล: `modules/mod4_genotype_phenotype_map.py (ส่วนที่ 3/5)`

โมดูลที่ 4 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4)

```
101 |             active_determinants.append("ampC")
102 |             mechanisms.append("AmpC beta-lactamase refractory to clavulanate inhibition")
103 |             genetic_score = 0.95
104 |             fold_increase = 8.0
105 |         if "blaOXA-1" in detected_genes:
106 |             active_determinants.append("blaOXA-1")
107 |             mechanisms.append("Class D OXA-1 beta-lactamase resistant to clavulanate")
108 |             genetic_score = max(genetic_score, 0.90)
109 |             fold_increase = max(fold_increase, 4.0)
110 |         if not active_determinants and (
111 |             "blaTEM-1" in detected_genes or "blaCTX-M-15" in detected_genes
112 |         ):
113 |             for g in ("blaTEM-1", "blaCTX-M-15"):
114 |                 if g in detected_genes:
115 |                     active_determinants.append(g)
116 |             mechanisms.append(
117 |                 "Class A beta-lactamase neutralized by clavulanate (borderline susceptible MIC)"
118 |             )
119 |             genetic_score = 0.35
120 |             fold_increase = 2.0
121 |
122 |     elif drug_code == "CIP":
123 |         gyra_muts = sorted(m for m in detected_mutations if m.startswith("gyrA_"))
124 |         parc_muts = sorted(m for m in detected_mutations if m.startswith("parC_"))
125 |         has_qnr = "qnrS1" in detected_genes
126 |
127 |         active_determinants.extend(gyra_muts + parc_muts)
128 |         if has_qnr:
129 |             active_determinants.append("qnrS1")
130 |
131 |         if len(gyra_muts) >= 2 and len(parc_muts) >= 1:
132 |             mechanisms.append(
133 |                 "Dual QRDR target alteration in DNA gyrase (GyrA) and topoisomerase IV (ParC)"
134 |             )
135 |             genetic_score = 0.98
136 |             fold_increase = 16.0
137 |         elif len(gyra_muts) >= 1 and has_qnr:
138 |             mechanisms.append(
139 |                 "Synergistic GyrA QRDR mutation plus QnrS1 target protection protein"
140 |             )
141 |             genetic_score = 0.86
142 |             fold_increase = 8.0
143 |         elif len(gyra_muts) >= 1 or has_qnr:
144 |             mechanisms.append("Single-step low-level quinolone susceptibility reduction")
145 |             genetic_score = 0.45
146 |             fold_increase = 4.0
147 |
148 |     elif drug_code == "NIT":
149 |         nfsa_muts = sorted(m for m in detected_mutations if m.startswith("nfsA_"))
150 |         nfsb_muts = sorted(m for m in detected_mutations if m.startswith("nfsB_"))
```

ไฟล์โมดูล: `modules/mod4_genotype_phenotype_map.py (ส่วนที่ 4/5)`

โมดูลที่ 4 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4)

```
151 |         active_determinants.extend(nfsa_muts + nfsb_muts)
152 |
153 |         if nfsa_muts and nfsb_muts:
154 |             has_nonsense = any(m.endswith("*") for m in nfsa_muts + nfsb_muts)
155 |             mechanisms.append(
156 |                 "Dual inactivation of oxygen-insensitive nitroreductases NfsA and NfsB preventing prodrug activation"
157 |             )
158 |             genetic_score = 0.97 if has_nonsense else 0.92
159 |             fold_increase = 16.0 if ("nfsA_R15*" in nfsa_muts) else 8.0
160 |         elif nfsa_muts or nfsb_muts:
161 |             mechanisms.append("Single nitroreductase step mutation (borderline MIC elevation)")
162 |             genetic_score = 0.40
163 |             fold_increase = 2.0
164 |
165 |     elif drug_code == "TET":
166 |         tet_genes = [g for g in ("tetA", "tetB") if g in detected_genes]
167 |         if tet_genes:
168 |             active_determinants.extend(tet_genes)
169 |             mechanisms.append("Major facilitator superfamily (MFS) active tetracycline efflux pump")
170 |             genetic_score = 0.95
171 |             fold_increase = 8.0
172 |
173 |     if not mechanisms:
174 |         mechanisms.append("Wild-type susceptible target / no resistance determinant detected")
175 |
176 |     combined_probability = round(
177 |         min(0.99, 0.85 * genetic_score + 0.15 * float(baseline_ns_rate)),
178 |         4,
179 |     )
180 |
181 |     return {
182 |         "drug_code": drug_code,
183 |         "drug_name": CLSI_DRUG_BREAKPOINTS[drug_code]["drug_name"],
184 |         "drug_class": CLSI_DRUG_BREAKPOINTS[drug_code]["drug_class"],
185 |         "active_determinants": active_determinants,
186 |         "molecular_mechanism": "; ".join(mechanisms),
187 |         "genetic_score": round(genetic_score, 4),
188 |         "baseline_clinical_ns_rate": round(float(baseline_ns_rate), 4),
189 |         "combined_ns_probability": combined_probability,
190 |         "estimated_fold_increase": fold_increase,
191 |     }
192 |
193 |
194 | def map_genetic_determinants_to_drugs(
195 |     scan_results: dict[str, dict[str, Any]],
196 |     clinical_profile: dict[str, Any],
197 | ) -> dict[str, dict[str, Any]]:
198 |     """
199 |     ฟังก์ชันย่อยหลักที่ 4: แปลงผลการตรวจหายีนดื้อยาและการกลายพันธุ์ของทุกสายพันธุ์จาก Module 3
200 |     ให้เป็นโปรไฟล์คะแนนกลไกการดื้อยาต่อยาปฏิชีวนะทั้ง 5 ชนิด โดยถ่วงน้ำหนักร่วมกับสถิติคลินิกจาก Module 1
```

ไฟล์โมดูล: `modules/mod4_genotype_phenotype_map.py (ส่วนที่ 5/5)`

โมดูลที่ 4 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 4 (6XXXXXXXX-4)

```
201 |
202 |     ข้อมูลเข้า (Input):
203 |         scan_results (dict[str, dict[str, Any]]): ผลการสแกนยีนดื้อยาและการกลายพันธุ์จาก Module 3
204 |         clinical_profile (dict[str, Any]): โปรไฟล์ระบาดวิทยาคลินิกจาก Module 1
205 |
206 |     การดำเนินงาน (Action):
207 |         1. ดึงค่า baseline non-susceptible rate ของยาแต่ละชนิดจาก clinical_profile
208 |         2. สำหรับแต่ละสายพันธุ์ เรียกใช้ evaluate_drug_mechanism_score() ครบทั้ง 5 ตัวยา (AMC, AMP, CIP, NIT, TET)
209 |         3. รวบรวมกลไกการดื้อยาระดับโมเลกุลและคะแนนความน่าจะเป็นส่งต่อให้ Module 5
210 |
211 |     ผลลัพธ์ (Output):
212 |         dict[str, dict[str, Any]]: แผนที่จาก isolate_id ไปยังคะแนนกลไกการดื้อยาของยาทั้ง 5 ชนิด
213 |     """
214 |     if not scan_results:
215 |         raise ValueError("scan_results ต้องไม่เป็นค่าว่าง")
216 |
217 |     drug_profiles = clinical_profile.get("drug_profiles", {})
218 |     mapped_isolates: dict[str, dict[str, Any]] = {}
219 |
220 |     for iso_id, scan_info in scan_results.items():
221 |         genes_set = set(scan_info.get("acquired_gene_names", []))
222 |         muts_set = set(scan_info.get("mutation_labels", []))
223 |
224 |         per_drug_scores: dict[str, dict[str, Any]] = {}
225 |         for drug_code in sorted(CLSI_DRUG_BREAKPOINTS):
226 |             baseline_ns = (
227 |                 drug_profiles.get(drug_code, {}).get("non_susceptible_rate", 0.20)
228 |             )
229 |             per_drug_scores[drug_code] = evaluate_drug_mechanism_score(
230 |                 drug_code=drug_code,
231 |                 detected_genes=genes_set,
232 |                 detected_mutations=muts_set,
233 |                 baseline_ns_rate=baseline_ns,
234 |             )
235 |
236 |         mapped_isolates[iso_id] = {
237 |             "isolate_id": iso_id,
238 |             "clinical_id": scan_info.get("clinical_id", "UNKNOWN"),
239 |             "strain": scan_info.get("strain", ""),
240 |             "acquired_gene_names": sorted(genes_set),
241 |             "mutation_labels": sorted(muts_set),
242 |             "drug_mechanism_scores": per_drug_scores,
243 |         }
244 |
245 |     return mapped_isolates
```

ไฟล์โมดูล: `modules/mod5_resistance_predictor.py (ส่วนที่ 1/6)`

โมดูลที่ 5 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

```
"""
Module 5: mod5_resistance_predictor.py
ฟังก์ชันย่อยที่ 5 — การทำนายฟีโนไทป์ความไวและการดื้อยา (S vs NS), การคาดการณ์ค่า MIC,
การจำแนกเชื้อดื้อยาหลายขนาน (MDR) และการประเมินความแม่นยำเทียบกับข้อมูลจริง (Sensitivity, Specificity, Accuracy)

ผู้รับผิดชอบ: สมาชิกคนที่ 5 (กลุ่ม 3)
"""

from __future__ import annotations

import csv
import json
from pathlib import Path
from typing import Any

from .mod4_genotype_phenotype_map import CLSI_DRUG_BREAKPOINTS


def classify_mic_and_phenotype(
    drug_code: str,
    mechanism_score_info: dict[str, Any],
) -> dict[str, Any]:
    """
    ทำนายค่าความเข้มข้นต่ำสุดที่ยับยั้งเชื้อได้ (Predicted MIC) และจำแนกฟีโนไทป์ความไวต่อยา (S หรือ NS)
    ตามเกณฑ์ CLSI Breakpoint

    ข้อมูลเข้า (Input):
        drug_code (str): รหัสยาปฏิชีวนะ (AMC, AMP, CIP, NIT, TET)
        mechanism_score_info (dict[str, Any]): ข้อมูลคะแนนกลไกการดื้อยาจาก Module 4

    การดำเนินงาน (Action):
        คำนวณค่า Predicted MIC จาก wildtype_mic คูณด้วย estimated_fold_increase
        และเปรียบเทียบกับ CLSI Breakpoint (ns_min_mic) ร่วมกับค่า combined_ns_probability >= 0.50

    ผลลัพธ์ (Output):
        dict[str, Any]: ค่า predicted_mic, predicted_phenotype ('S' หรือ 'NS'), confidence_score และกลไกที่พบ
    """
    if drug_code not in CLSI_DRUG_BREAKPOINTS:
        raise ValueError(f"ไม่รองรับรหัสยา: {drug_code}")

    bp = CLSI_DRUG_BREAKPOINTS[drug_code]
    fold = float(mechanism_score_info.get("estimated_fold_increase", 1.0))
    prob_ns = float(mechanism_score_info.get("combined_ns_probability", 0.0))

    predicted_mic = round(bp["wildtype_mic"] * fold, 4)
    is_non_susceptible = (predicted_mic >= bp["ns_min_mic"]) or (prob_ns >= 0.50)
    predicted_phenotype = "NS" if is_non_susceptible else "S"
    confidence = prob_ns if is_non_susceptible else round(1.0 - prob_ns, 4)

    return {
```

ไฟล์โมดูล: `modules/mod5_resistance_predictor.py (ส่วนที่ 2/6)`

โมดูลที่ 5 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

```
 51 |         "drug_code": drug_code,
 52 |         "drug_name": bp["drug_name"],
 53 |         "drug_class": bp["drug_class"],
 54 |         "predicted_mic": predicted_mic,
 55 |         "predicted_phenotype": predicted_phenotype,
 56 |         "ns_probability": round(prob_ns, 4),
 57 |         "prediction_confidence": round(confidence, 4),
 58 |         "active_determinants": mechanism_score_info.get("active_determinants", []),
 59 |         "molecular_mechanism": mechanism_score_info.get("molecular_mechanism", ""),
 60 |     }
 61 |
 62 |
 63 | def compute_confusion_and_concordance(
 64 |     comparison_rows: list[dict[str, Any]],
 65 | ) -> dict[str, Any]:
 66 |     """
 67 |     คำนวณเมทริกซ์ความสับสน (Confusion Matrix: TP, TN, FP, FN) และค่าประสิทธิภาพการทำนาย
 68 |     ได้แก่ Accuracy, Sensitivity (Recall ของ NS), Specificity (Recall ของ S), Precision และ F1-Score
 69 |
 70 |     ข้อมูลเข้า (Input):
 71 |         comparison_rows (list[dict[str, Any]]): รายการผลการทำนายคู่กับผลฟีโนไทป์จริง (actual_phenotype, predicted_phenotype)
 72 |
 73 |     การดำเนินงาน (Action):
 74 |         นับจำนวน TP (จริง NS ทำนาย NS), TN (จริง S ทำนาย S), FP (จริง S ทำนาย NS), FN (จริง NS ทำนาย S)
 75 |         ทั้งในภาพรวมและแยกตามชนิดยาปฏิชีวนะ
 76 |
 77 |     ผลลัพธ์ (Output):
 78 |         dict[str, Any]: ค่าสถิติความแม่นยำรวม (overall) และแยกตามตัวยา (per_drug)
 79 |     """
 80 |     if not comparison_rows:
 81 |         raise ValueError("comparison_rows ต้องมีข้อมูลอย่างน้อย 1 รายการ")
 82 |
 83 |     def _calc_metrics(rows: list[dict[str, Any]]) -> dict[str, Any]:
 84 |         tp = sum(
 85 |             1
 86 |             for r in rows
 87 |             if r["actual_phenotype"] == "NS" and r["predicted_phenotype"] == "NS"
 88 |         )
 89 |         tn = sum(
 90 |             1
 91 |             for r in rows
 92 |             if r["actual_phenotype"] == "S" and r["predicted_phenotype"] == "S"
 93 |         )
 94 |         fp = sum(
 95 |             1
 96 |             for r in rows
 97 |             if r["actual_phenotype"] == "S" and r["predicted_phenotype"] == "NS"
 98 |         )
 99 |         fn = sum(
100 |             1
```

ไฟล์โมดูล: `modules/mod5_resistance_predictor.py (ส่วนที่ 3/6)`

โมดูลที่ 5 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

```
101 |             for r in rows
102 |             if r["actual_phenotype"] == "NS" and r["predicted_phenotype"] == "S"
103 |         )
104 |         total = tp + tn + fp + fn
105 |         accuracy = (tp + tn) / total if total else 0.0
106 |         sensitivity = tp / (tp + fn) if (tp + fn) else 0.0
107 |         specificity = tn / (tn + fp) if (tn + fp) else 0.0
108 |         precision = tp / (tp + fp) if (tp + fp) else 0.0
109 |         f1 = (
110 |             (2 * precision * sensitivity / (precision + sensitivity))
111 |             if (precision + sensitivity)
112 |             else 0.0
113 |         )
114 |         exact_mic_matches = sum(
115 |             1
116 |             for r in rows
117 |             if r.get("actual_mic") is not None
118 |             and abs(float(r["predicted_mic"]) - float(r["actual_mic"])) <= 1e-6
119 |         )
120 |         return {
121 |             "total_comparisons": total,
122 |             "true_positive_ns": tp,
123 |             "true_negative_s": tn,
124 |             "false_positive_ns": fp,
125 |             "false_negative_s": fn,
126 |             "accuracy": round(accuracy, 4),
127 |             "sensitivity_ns": round(sensitivity, 4),
128 |             "specificity_s": round(specificity, 4),
129 |             "precision_ns": round(precision, 4),
130 |             "f1_score_ns": round(f1, 4),
131 |             "exact_mic_concordance_rate": round(exact_mic_matches / total, 4)
132 |             if total
133 |             else 0.0,
134 |         }
135 |
136 |     overall_metrics = _calc_metrics(comparison_rows)
137 |     per_drug_metrics: dict[str, Any] = {}
138 |     for drug_code in sorted(CLSI_DRUG_BREAKPOINTS):
139 |         d_rows = [r for r in comparison_rows if r["drug_code"] == drug_code]
140 |         if d_rows:
141 |             per_drug_metrics[drug_code] = _calc_metrics(d_rows)
142 |
143 |     return {
144 |         "overall": overall_metrics,
145 |         "per_drug": per_drug_metrics,
146 |     }
147 |
148 |
149 | def predict_susceptibility_and_evaluate(
150 |     mapped_profiles: dict[str, dict[str, Any]],
```

ไฟล์โมดูล: `modules/mod5_resistance_predictor.py (ส่วนที่ 4/6)`

โมดูลที่ 5 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

```
151 |     clinical_profile: dict[str, Any],
152 |     isolate_genomes: list[dict[str, Any]],
153 |     output_dir: str | Path,
154 | ) -> dict[str, Any]:
155 |     """
156 |     ฟังก์ชันย่อยหลักที่ 5: ทำนายผลความไวและการดื้อยาปฏิชีวนะของสายพันธุ์ E. coli ทั้งหมด
157 |     จำแนกสายพันธุ์ดื้อยาหลายขนาน (MDR), ประเมินความแม่นยำเทียบกับค่าจริง และส่งออกไฟล์ CSV และ JSON
158 |
159 |     ข้อมูลเข้า (Input):
160 |         mapped_profiles (dict[str, dict[str, Any]]): ข้อมูลคะแนนกลไกดื้อยาจาก Module 4
161 |         clinical_profile (dict[str, Any]): โปรไฟล์สถิติคลินิกจาก Module 1
162 |         isolate_genomes (list[dict[str, Any]]): ข้อมูลจีโนมและฟีโนไทป์จริงของสายพันธุ์จาก Module 2
163 |         output_dir (str | Path): โฟลเดอร์ปลายทางสำหรับบันทึกไฟล์ผลลัพธ์ CSV และ JSON
164 |
165 |     การดำเนินงาน (Action):
166 |         1. เรียกใช้ classify_mic_and_phenotype() ทำนายค่า MIC และ S/NS ของแต่ละสายพันธุ์ต่อยาทั้ง 5 ชนิด
167 |         2. ระบุสถานะ MDR (ดื้อยา >= 3 กลุ่มยาปฏิชีวนะ) ของแต่ละสายพันธุ์
168 |         3. เรียกใช้ compute_confusion_and_concordance() เพื่อวัดประสิทธิภาพของโมเดลทำนาย
169 |         4. บันทึกตาราง amr_predictions.csv และรายงาน project2_evaluation_summary.json
170 |
171 |     ผลลัพธ์ (Output):
172 |         dict[str, Any]: รายงานสรุปผลการทำนาย ความแม่นยำ และพาธของไฟล์ผลลัพธ์ที่สร้างขึ้น
173 |     """
174 |     out_path = Path(output_dir)
175 |     out_path.mkdir(parents=True, exist_ok=True)
176 |
177 |     meta_by_id = {iso["isolate_id"]: iso for iso in isolate_genomes}
178 |     prediction_rows: list[dict[str, Any]] = []
179 |     isolate_summaries: list[dict[str, Any]] = []
180 |
181 |     for iso_id in sorted(mapped_profiles):
182 |         prof = mapped_profiles[iso_id]
183 |         iso_meta = meta_by_id.get(iso_id, {})
184 |         known_phenos = iso_meta.get("known_phenotypes", {})
185 |
186 |         ns_drug_classes: set[str] = set()
187 |         ns_drugs: list[str] = []
188 |
189 |         for drug_code in sorted(CLSI_DRUG_BREAKPOINTS):
190 |             score_info = prof["drug_mechanism_scores"][drug_code]
191 |             pred = classify_mic_and_phenotype(drug_code, score_info)
192 |
193 |             actual_info = known_phenos.get(drug_code, {})
194 |             actual_pheno = actual_info.get("phenotype", "UNKNOWN")
195 |             actual_mic = actual_info.get("MIC")
196 |
197 |             if pred["predicted_phenotype"] == "NS":
198 |                 ns_drug_classes.add(pred["drug_class"])
199 |                 ns_drugs.append(drug_code)
200 |
```

ไฟล์โมดูล: `modules/mod5_resistance_predictor.py (ส่วนที่ 5/6)`

โมดูลที่ 5 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

```
201 |             row_record = {
202 |                 "isolate_id": iso_id,
203 |                 "clinical_id": prof.get("clinical_id", "UNKNOWN"),
204 |                 "strain": prof.get("strain", ""),
205 |                 "ncbi_assembly": iso_meta.get("ncbi_assembly", "N/A"),
206 |                 "ena_accession": iso_meta.get("ena_accession", "N/A"),
207 |                 "drug_code": drug_code,
208 |                 "drug_name": pred["drug_name"],
209 |                 "drug_class": pred["drug_class"],
210 |                 "predicted_mic": pred["predicted_mic"],
211 |                 "actual_mic": actual_mic,
212 |                 "predicted_phenotype": pred["predicted_phenotype"],
213 |                 "actual_phenotype": actual_pheno,
214 |                 "concordant": pred["predicted_phenotype"] == actual_pheno,
215 |                 "ns_probability": pred["ns_probability"],
216 |                 "active_determinants": " | ".join(pred["active_determinants"])
217 |                 if pred["active_determinants"]
218 |                 else "None (Wild-type)",
219 |                 "molecular_mechanism": pred["molecular_mechanism"],
220 |             }
221 |             prediction_rows.append(row_record)
222 |
223 |         is_mdr = len(ns_drug_classes) >= 3
224 |         isolate_summaries.append(
225 |             {
226 |                 "isolate_id": iso_id,
227 |                 "clinical_id": prof.get("clinical_id", "UNKNOWN"),
228 |                 "strain": prof.get("strain", ""),
229 |                 "ncbi_assembly": iso_meta.get("ncbi_assembly", "N/A"),
230 |                 "ena_accession": iso_meta.get("ena_accession", "N/A"),
231 |                 "gc_percent": iso_meta.get("qc_metrics", {}).get("gc_percent", 0.0),
232 |                 "genome_length_bp": iso_meta.get("qc_metrics", {}).get("length_bp", 0),
233 |                 "acquired_genes": prof.get("acquired_gene_names", []),
234 |                 "point_mutations": prof.get("mutation_labels", []),
235 |                 "predicted_ns_drugs": ns_drugs,
236 |                 "predicted_ns_class_count": len(ns_drug_classes),
237 |                 "predicted_mdr": is_mdr,
238 |             }
239 |         )
240 |
241 |     evaluable_rows = [
242 |         r for r in prediction_rows if r["actual_phenotype"] in ("S", "NS")
243 |     ]
244 |     evaluation_metrics = compute_confusion_and_concordance(evaluable_rows)
245 |
246 |     csv_file = out_path / "amr_predictions.csv"
247 |     fieldnames = [
248 |         "isolate_id",
249 |         "clinical_id",
250 |         "strain",
```

ไฟล์โมดูล: `modules/mod5_resistance_predictor.py (ส่วนที่ 6/6)`

โมดูลที่ 5 — ผู้รับผิดชอบ: ชื่อ-นามสกุล สมาชิกคนที่ 5 (6XXXXXXXX-5)

```
251 |         "ncbi_assembly",
252 |         "ena_accession",
253 |         "drug_code",
254 |         "drug_name",
255 |         "drug_class",
256 |         "predicted_mic",
257 |         "actual_mic",
258 |         "predicted_phenotype",
259 |         "actual_phenotype",
260 |         "concordant",
261 |         "ns_probability",
262 |         "active_determinants",
263 |         "molecular_mechanism",
264 |     ]
265 |     with csv_file.open("w", encoding="utf-8-sig", newline="") as handle:
266 |         writer = csv.DictWriter(handle, fieldnames=fieldnames)
267 |         writer.writeheader()
268 |         writer.writerows(prediction_rows)
269 |
270 |     json_file = out_path / "project2_evaluation_summary.json"
271 |     summary_payload = {
272 |         "project": "Bioinformatics Programming Project II — Group 3",
273 |         "topic": "Predicting Antibiotic Susceptibility and Resistance in Escherichia coli from Genomic Determinants and
Clinical Phenotypes",
274 |         "clinical_dataset_overview": {
275 |             "total_records_analyzed": clinical_profile.get("total_records_analyzed", 0),
276 |             "unique_clinical_isolates": clinical_profile.get("unique_isolates", 0),
277 |             "clinical_mdr_summary": clinical_profile.get("mdr_summary", {}),
278 |             "drug_profiles": clinical_profile.get("drug_profiles", {}),
279 |         },
280 |         "genomic_cohort_overview": {
281 |             "total_isolates_sequenced": len(isolate_summaries),
282 |             "total_drug_test_predictions": len(prediction_rows),
283 |             "predicted_mdr_isolates": sum(
284 |                 1 for s in isolate_summaries if s["predicted_mdr"]
285 |             ),
286 |             "isolate_summaries": isolate_summaries,
287 |         },
288 |         "evaluation_metrics": evaluation_metrics,
289 |         "output_files": {
290 |             "predictions_csv": str(csv_file),
291 |             "summary_json": str(json_file),
292 |         },
293 |     }
294 |     json_file.write_text(
295 |         json.dumps(summary_payload, indent=2, ensure_ascii=False) + "\n",
296 |         encoding="utf-8",
297 |     )
298 |
299 |     return summary_payload
```